



WYDZIAŁ ELEKTRONIKI,
TELEKOMUNIKACJI
I INFORMATYKI

Imię i nazwisko studenta: Jakub Ławicki

Nr albumu: 189788

Poziom kształcenia: studia drugiego stopnia

Forma studiów: stacjonarne

Kierunek studiów: Elektronika i telekomunikacja

Specjalność: Systemy wbudowane czasu rzeczywistego

PRACA DYPLMOWA MAGISTERSKA

Tytuł pracy w języku polskim: Badanie wpływu różnych metod optymalizacji kodu na wydajność systemu wbudowanego opartego na systemie Linux.

Tytuł pracy w języku angielskim: Investigation of the impact of various code optimization methods on the performance of a Linux-based embedded system.

Opiekun pracy: dr hab. inż. Jacek Marszał

STRESZCZENIE

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec odio sapien, egestas a ullamcorper a, volutpat tempor felis. Proin vel laoreet augue, a tincidunt ex. Nam maximus massa nibh, ac convallis nunc viverra ac. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Maecenas felis mi, venenatis id iaculis eget, aliquam sit amet purus. Ut vehicula convallis finibus. Nam vulputate commodo magna, eu dapibus odio laoreet in. Phasellus tempus bibendum elementum. Pellentesque dapibus pulvinar orci nec sodales. Etiam eu augue non ex sodales sollicitudin vitae quis mi. Nulla vitae elit ac nisl egestas tempor. Nam ut nisl gravida, convallis purus sed, dapibus augue.

Słowa kluczowe: Optymalizacja algorytmów, systemy wbudowane, system operacyjny Linux

Dziedzina nauki i techniki zgodna z OECD Nauki inżynieryjne i techniczne, Elektrotechnika, elektronika i inżynieria informatyczna,

ABSTRACT

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec odio sapien, egestas a ullamcorper a, volutpat tempor felis. Proin vel laoreet augue, a tincidunt ex. Nam maximus massa nibh, ac convallis nunc viverra ac. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Maecenas felis mi, venenatis id iaculis eget, aliquam sit amet purus. Ut vehicula convallis finibus. Nam vulputate commodo magna, eu dapibus odio laoreet in. Phasellus tempus bibendum elementum. Pellentesque dapibus pulvinar orci nec sodales. Etiam eu augue non ex sodales sollicitudin vitae quis mi. Nulla vitae elit ac nisl egestas tempor. Nam ut nisl gravida, convallis purus sed, dapibus augue.

Keywords: Self-organizing maps, unsupervised learning, clustering, classification, WTM algorithm

OECD consistent field of science and technology classification: Engineering and technology, Electrical engineering, Electronic engineering, Information engineering,

SPIS TREŚCI

Spis treści	5
1 Wstęp i cel pracy	8
1.1 Cel pracy	8
1.2 Zawartość rozdziałów	8
2 Architektura systemów operacyjnych w zastosowaniach wbudowanych	9
2.1 Systemy wbudowane czasu rzeczywistego wraz z ich podziałem	9
2.2 Rola systemów operacyjnych w systemach wbudowanych	11
2.2.1 Model warstwowy systemów operacyjnych	12
2.2.2 Podział systemów operacyjnych ze względu na strukturę systemu	13
2.2.3 Korzyści i niedogodności płynące z stosowania systemów operacyjnych	14
2.3 Studium przypadku systemu Linux	14
2.3.1 Architektura systemu i jego jądro	14
2.3.2 Zarządzanie pamięcią	14
2.3.3 Procesy i wątki w systemie Linux	14
2.3.4 Szeregowani zadań i determinizm czasowy w systemie Linux	14
2.3.5 Wykorzystanie systemu Linux w systemach wbudowanych	14
3 Wybrany algorytm przetwarzania danych i techniki optymalizacji kodu	15
3.1 Teoretyczne podstawy analizy algorytmów	15
3.2 Opis i analiza Szybkiej Transformaty Fouriera	15
3.3 Warianty algorytmu Szybkiej Transformaty Fouriera	15
3.4 Przetworzenie kodu programu komputerowego na kod wykonywalny	15
3.5 Opis kompilacji programów w języku C	15
3.6 Poziomy i flagi optymalizacyjne kompilatora GCC	15
3.7 Zaawansowane techniki optymalizacji kodu	15
4 Techniki optymalizacji programów komputerowych	16
4.1 Opis procesu kompilacji programu komputerowego	16
4.2 Programowe techniki optymalizacji programów komputerowych	16
4.3 Przegląd najpopularniejszych kompilatorów języka C	16
4.4 Poziomy optymalizacji dostępne w kompilatorach Clang i Gcc	16
4.5 Weryfikacja poprawności obliczeń z biblioteką zewnętrzną	16
5 Projekt systemu testowego i metodyka przeprowadzonych badań	17
5.1 Założenia projektowe i architektura aplikacji	17
5.2 Opis platformy sprzętowej	17
5.3 Użyte narzędzia programistyczne	17
5.4 Metodyka przeprowadzonych pomiarów	17
5.5 Prezentacja przygotowywanych wariantów programu	17
5.6 Implementacja wersji wielowątkowej, opis biblioteki pthread	17
5.7 Weryfikacja poprawności obliczeń z biblioteką zewnętrzną	17

6	Analiza wyników przeprowadzonych badań	18
6.1	Analiza wpływu flag kompilatora na kod sekwencyjny	18
6.2	Analiza wpływu optymalizacji ręcznej programu sekwencyjnego	18
6.3	Analiza skalowalności rozwiązania wielowątkowego	18
6.4	Zestawienie zbiorcze i dyskusja wyników	18
7	Podsumowanie i wnioski	19
7.1	Wnioski końcowe	19
7.2	kierunek dalszych badań	19
	Spis rysunków	20
	Spis tabel	21

LISTA SYMBOLI

LISTA SKRÓTÓW

1. WSTĘP I CEL PRACY

1.1. Cel pracy

1.2. Zawartość rozdziałów

Ogólną strukturę niniejszej pracy można przedstawić w następujący sposób:

- **Rozdział 2: Charakterystyka systemów wbudowanych opartych na systemie Linux.**
Obejmuje charakterystykę systemów wbudowanych opartych na systemie Linux, z opisem podstawowych pojęć jak jądro systemu, menadżer procesów, zarządzanie pamięcią oraz mechanizmy wielozadaniowości. Opisuje specyfikę zarządzania zasobami sprzętowymi przez jądro systemu. Szczególną uwagę poświęcono mechanizmom wielozadaniowości, szeregowania wątków oraz zarządzania pamięcią wirtualną, które mają kluczowy wpływ na wydajność aplikacji.
- **Rozdział 3: Wybrany algorytm przetwarzania danych i techniki optymalizacji kodu.**
Przedstawia teoretyczne podstawy algorytmu Szybkiej Transformaty Fouriera (FFT) oraz analizę jego złożoności obliczeniowej. Prezentuje różne implementacje tego algorytmu. Omawia proces kompilacji kodu w języku C, nakreślając istotę wyboru kompilatora. Ponadto przedstawia poziomy optymalizacji oferowane przez kompilator GCC, używany w pracy. Wyjaśnia zaawansowane techniki programistyczne, takie jak instrukcje wektorowe oraz optymalizacja pod kątem dostępu do pamięci podręcznej.
- **Rozdział 4: Projekt i implementacja aplikacji testowej.**
Opisuje szczegółowo opracowane oprogramowanie realizujące algorytm Szybkiej Transformaty Fouriera (FFT). Przedstawia alternatywne warianty implementacji kodu: wersję sekwencyjną, wersję zoptymalizowaną pod kątem dostępu do pamięci oraz wersję wielowątkową wykorzystującą mechanizmy współbieżności.
- **Rozdział 5: Środowisko badawcze i metodyka pomiarów.**
Charakteryzuje wykorzystaną platformę sprzętową oraz konfigurację systemu operacyjnego Linux. Opisuje narzędzia programistyczne, biblioteki oraz frameworki użyte do implementacji i optymalizacji kodu. Definiuje scenariusze testowe, przygotowane konfiguracje procesu kompilacji (zestawy flag optymalizacyjnych) oraz narzędzia i metody użyte do pomiaru czasu wykonania i zużycia zasobów systemowych.
- **Rozdział 6: Analiza wpływu optymalizacji na wydajność systemu.**
Prezentuje wyniki przeprowadzonych badań. Każdy podrozdział zawiera zestawienie i analizę wpływu konkretnych czynników – poziomu optymalizacji kompilatora, zmian w kodzie źródłowym oraz liczby wątków – na ostateczną wydajność aplikacji w systemie wbudowanym.
- **Rozdział 7: Podsumowanie i wnioski.**
Stanowi syntezę uzyskanych wyników oraz ocenę skuteczności badanych metod optymalizacji. Zawiera wnioski końcowe dotyczące relacji między nakładem pracy programisty a automatyczną optymalizacją kompilatora oraz propozycje dalszych kierunków badań.

2. ARCHITEKTURA SYSTEMÓW OPERACYJNYCH W ZASTOSOWANIACH WBUDOWANYCH

Współczesna inżynieria systemów wbudowanych znajduje się w punkcie zwrotnym, w którym tradycyjne podejście oparte na bezpośrednim sterowaniu sprzętem (Bare-Metal) ustępuje miejsca zaawansowanym platformom zintegrowanym z systemami operacyjnymi klasy Linux. Ta ewolucja jest podyktowana rosnącą złożonością realizowanych zadań, wymagających nie tylko wysokiej mocy obliczeniowej, ale również sprawnego zarządzania współbieżnością, pamięcią wirtualną oraz interfejsami komunikacyjnymi.

System operacyjny, zgodnie z klasyczną definicją A. Tanenbauma, pełni w tym układzie rolę zarządcy zasobów, którego zadaniem jest sprawiedliwy i bezpieczny arbitraż dostępu do fizycznych komponentów platformy. Jednakże, w przypadku algorytmów o wysokiej intensywności obliczeniowej, takich jak Szybka Transformata Fouriera (FFT), ta warstwa abstrakcji staje się źródłem specyficznych wyzwań inżynierskich. Mechanizmy jądra, takie jak szeregowanie zadań czy obsługa przerw, wprowadzają do systemu niedeterminizm, objawiający się fluktuacjami czasu wykonania kodu (jitterem) oraz narzutem administracyjnym (overhead).

Zrozumienie architektury systemu operacyjnego w zastosowaniach wbudowanych jest zatem kluczowe dla procesu optymalizacji. Niniejszy rozdział stanowi fundament teoretyczny dla dalszych badań, analizując drogę danych od fizycznego sygnału, przez mechanizmy izolacji przestrzeni jądra i użytkownika, aż po polityki planisty systemowego. Celem poniższych rozważań jest wskazanie wąskich gardeł systemowych, które mogą ograniczać teoretyczną wydajność algorytmów cyfrowego przetwarzania sygnałów na platformach o ograniczonych zasobach

[1]

2.1. Systemy wbudowane czasu rzeczywistego wraz z ich podziałem

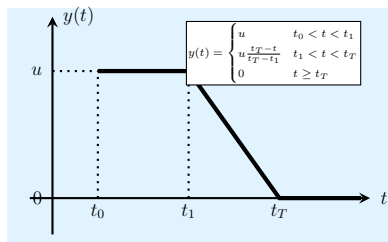
Systemem wbudowanym nazywamy dedykowany system komputerowy, który stanowi integralną część większego urządzenia lub maszyny i jest zaprojektowany do wykonywania ściśle określonego zadania [1]. W przeciwieństwie do komputerów ogólnego przeznaczenia (takich jak komputery osobiste czy smartfony), na których użytkownik może uruchamiać dowolne aplikacje, system wbudowany ma z góry zdefiniowaną funkcję i nie jest przeznaczony do późniejszych, swobodnych modyfikacji przez użytkownika końcowego.

Szczególną grupą w obrębie systemów wbudowanych są **systemy wbudowane czasu rzeczywistego**. Są to takie systemy, których poprawność wyniku przetwarzania nie zależy tylko od poprawności logicznej odpowiedzi, ale też od czasu przetwarzania, poprzedzającego odpowiedź systemu [2]. W przypadku niewypracowania odpowiedzi w zadanym czasie zdarzenie to jest traktowane jako błąd systemu, co w skrajnych przypadkach może prowadzić do awarii lub uszkodzenia fizycznego sprzętu.

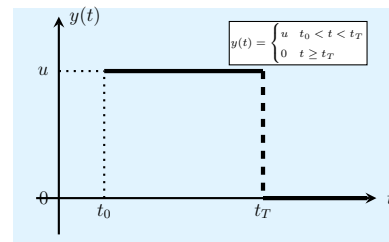
Systemy czasu rzeczywistego są powszechnie stosowane w zastosowaniach krytycznych, takich jak przemysł zbrojeniowy, aparatura medyczna czy branża motoryzacyjna, co wymaga wprowadzenia ścisłych kryteriów oceny ich niezawodności. W celu jednoznacznego zdefiniowania wymagań dotyczących determinizmu czasowego, w literaturze wprowadza się klasyfikacje systemów ze względu na rygor zachowania narzuconych terminów. W literaturze [1] systemy czasu rzeczywistego dzieli się na:

- Systemy twarde (Hard real-time systems)
- Systemy miękkie (Soft real-time systems)
- Systemy stabilne (Firm real-time systems)

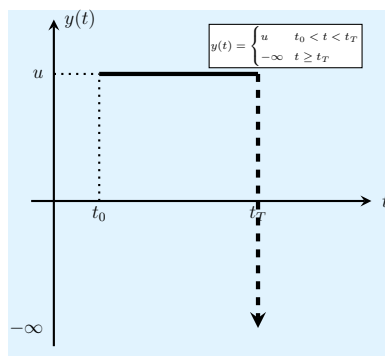
W celu lepszego zobrazowania podziału systemów czasu rzeczywistego, na rysunku rys. 2.1 przedstawiono wykresy funkcji przydatności odpowiedzi $y(t)$ od czasu jej wypracowania przez system [1].



(a) Miękki system czasu rzeczywistego



(b) Solidny system czasu rzeczywistego



(c) Twardy system czasu rzeczywistego

Rysunek 2.1: Zestawienie przebiegów funkcji przydatności odpowiedzi $y(t)$ w dziedzinie czasu [1].

Przed przystąpieniem do analizy poszczególnych wykresów, konieczne jest zdefiniowanie parametrów oraz skrótów stanowiących zmienne opisujące te funkcje:

- t – oś czasu, reprezentująca moment dostarczenia odpowiedzi przez system,
- $y(t)$ – funkcja przydatności (wartości) odpowiedzi dla obsługiwanego procesu,
- u – maksymalna, nominalna przydatność prawidłowo wypracowanego wyniku,
- t_0 – moment pojawienia się zdarzenia lub żądania wypracowania odpowiedzi przez system,
- t_1 – czas graniczny, do którego odpowiedź zachowuje pełną wartość (w systemach miękkich),
- t_T – ostateczny limit czasowy (ang. *deadline*), po przekroczeniu którego odpowiedź nie ma już pełnej wartości.

Analizując przebiegi funkcji przydatności dla poszczególnych klas systemów, można precyzyjnie scharakteryzować ich zachowanie.

Najbardziej restrykcyjne kryteria czasowe są charakterystyczne dla twardych systemy czasu rzeczywistego, co zobrazowano na rysunku 2.1c. Ich stosowanie gwarantuje bezwzględnie terminowe wypełnienie zadań krytycznych przed upływem limitu t_T . Przekroczenie czasu przetwarzania powoduje natychmiastowy spadek funkcji przydatności do wartości $-\infty$. W przypadku rozwiązań spóźniona odpowiedź jest nie tylko całkowicie bezużyteczna, ale jej pojawienie się po wyznaczonym terminie jest najczęściej równoznaczne z wystąpieniem krytycznego błędu, który może doprowadzić do poważnej awarii lub fizycznego uszkodzenia całego systemu.

Kolejną z omawianych grup stanowią miękkie systemy czasu rzeczywistego, których zachowanie zilustrowano na rysunku 2.1a. Systemy te cechują się mniejszym rygorem w kwestii terminowości, co oznacza, że zadania są wykonywane tak szybko, jak to możliwe, a ewentualne opóźnienia nie niosą za sobą skutków katastrofalnych. Jeśli odpowiedź zostanie dostarczona po czasie t_1 , ale przed ostatecznym limitem t_T , jej przydatność maleje, powodując jedynie spadek jej jakości. Przy przetwarzaniu przekraczającym czas t_T przydatność odpowiedzi wynosi zero.

Solidne systemy czasu rzeczywistego stanowią pewnego rodzaju kompromis pomiędzy twardymi, a miękkimi systemami czasu rzeczywistego. Zachowanie funkcji przydatności systemów solidnych przedstawiono na rysunku 2.1b. Każda odpowiedź dostarczona w przedziale od t_0 do t_T zachowuje stałą, pełną wartość u . W momencie przekroczenia terminu granicznego t_T przydatność wyniku natychmiastowo i skokowo spada do zera. Choć fakt spóźnienia powoduje całkowitą nieprzydatność rozwiązania, to sytuacja ta wciąż nie generuje bezpośredniego zagrożenia dla stabilności czy integralności samego systemu.

Zdefiniowane powyżej rygory czasowe określają, teoretyczny oczekiwania wobec systemu czasu rzeczywistego. W praktyce realizacja tych założeń sprowadza się do oceny zasadności stosowania **systemu operacyjnego czasu rzeczywistego** w zderzeniu z bezpośrednim programowaniem sprzętu. Decyzja ta bezpośrednio determinuje strukturę i elastyczność budowanego środowiska.

2.2. Rola systemów operacyjnych w systemach wbudowanych

Podczas projektowania systemu wbudowanego, włączając w to system wbudowany czasu rzeczywistego, projektant staje przed wyborem jednego z dwóch podstawowych podejść do organizacji i kontroli całego środowiska. Pierwszym z nich jest **podejście bezsystemowe** (ang. *bare-metal*), w którym całe oprogramowanie funkcjonuje bez żadnej warstwy pośredniej, mając bezpośredni i wyłączny dostęp do fizycznych układów procesora oraz peryferiów. Choć pozwala to na pełną kontrolę nad sprzętem i minimalne narzuty czasowe, to drastycznie ogranicza skalowalność każda zmiana komponentu lub rozbudowa funkcji wymusza wówczas głęboką modyfikację struktury całego kodu.

Alternatywą jest zastosowanie **systemu operacyjnego**, który możemy zdefiniować jako podstawowe oprogramowanie pośredniczące między sprzętem komputerowym a użytkownikiem, którego głównym zadaniem jest zarządzanie i przydzielanie zasobów sprzętowych oraz tworzenie środowiska pozwalającego na wykonywanie programów zaimplementowanych przez jego użytkownika [3].

Jak wynika z przytoczonej definicji, stosowanie systemu operacyjnego pomaga rozwiązywać niektóre problemy napotykane przy stosowaniu podejścia bezsystemowego. Dzieje się tak poprzez całościowe odizolowanie warstwy programowej od fizycznych struktur sprzętu. System operacyjny działa jako nadrzędny pośrednik, przejmując na siebie zadanie nadzorowania środowiska wykonawczego i separując logikę działania od konkretnej platformy sprzętowej.

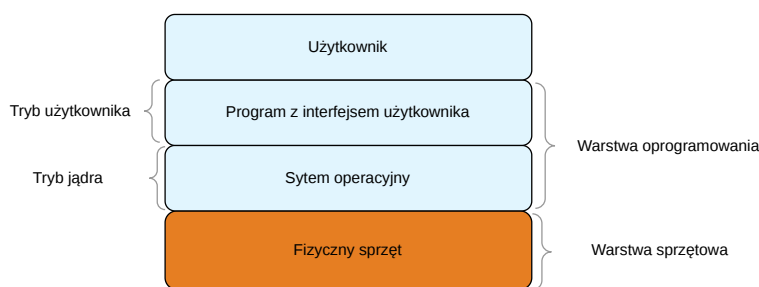
Aby w pełni zrozumieć, jak system operacyjny realizuje te zadania w środowisku wbudowanym, konieczne jest przyjrzenie się jego wewnętrznej architekturze. W dalszych sekcjach zostaną szczegółowo omówione kluczowe mechanizmy oraz podstawowa struktura ogólnych systemów operacyjnych.

2.2.1. Model warstwowy systemów operacyjnych

W inżynierii oprogramowania istotną rolę odgrywa koncepcja **abstrakcji**, polegająca na celowym ukryciu szczegółów implementacji danej funkcjonalności oprogramowania, przed jej potencjalnym użytkownikiem [1]. Podejście to opiera się na założeniu, że należy dostarczyć gotowe funkcjonalność wraz z odpowiednią dokumentacją opisującą sposób jej wykorzystania, zamiast wymagać od użytkownika zrozumienia wewnętrznych mechanizmów działania wytworzonego modułu.

Taka praktyka w sposób znaczący podnosi wygodę wytwarzania nowych elementów systemu, umożliwia zrównoleglenie wytwarzania oprogramowania oraz pozwala na większą specjalizację jej producentów. W celu realizacji komunikacji między kolejnymi modułami systemu stosuje się takie narzędzia jak **interfejsy programistyczne** (ang. *Application Programming Interface*, skrót **API**), czyli zbiór ściśle określonych reguł i sposobów komunikacji, które umożliwiają komunikację między różnymi warstwami oprogramowania. Narzędzie to jest niezbędne, aby w sposób przewidywalny i bezpieczny przekazywać dane lub sterowanie, między różnymi elementami systemu.

Pojęcie abstrakcji jest kluczowe dla zrozumienia architektury współczesnych systemów operacyjnych. W literaturze spotyka się różne podejścia do klasyfikacji poziomów systemu komputerowego [1, 3], jednak ich cechą wspólną jest wyraźne odseparowanie zasobów fizycznych od uruchamianego na nich oprogramowania. Na potrzeby niniejszej pracy przyjęto strukturę opartą na propozycji przedstawionej w [3], którą w graficzny sposób zaprezentowano na rysunku 2.2.



Rysunek 2.2: Warstwowy model systemu komputerowego

Jak pokazano na schemacie, najniższy poziom stanowi warstwa sprzętowa, do której zalicza się wszystkie fizyczne komponenty urządzenia, takie jak **jednostka centralna** (ang. *Central Processing Unit*, CPU), **pamięć** oraz **urządzenia wejścia-wyjścia** (ang. *Input/Output Devices*, I/O). Ponad nią znajduje się warstwa oprogramowania, na którą składa się zarówno system operacyjny, jak i programy użytkownika. Choć oba te elementy należą do tej samej warstwy, funkcjonują one na innych zasadach. Odróżnia je praca w różnych trybach procesora, różniących się poziomem uprawnień oraz zakresem dostępu do fizycznych urządzeń. Tryby te nazywają się odpowiednio **trybem jądra** (ang. *kernel mode*), dla systemu operacyjnego, oraz **trybem użytkownika** (ang. *user mode*), dla aplikacji użytkownika. Szczegółowa omówienie przytoczonych trybów oraz mechanizmów ich przełączania zostanie omówiona w dalszych częściach tego rozdziału.

Warto przy tym podkreślić, że w przytoczonej pozycji literatury [3] jako integralny komponent struktury systemu komputerowego uwzględnia się jego użytkownika. Zabieg ten ma na celu jednoznaczne wskazanie, że przy projektowaniu dowolnego systemu komputerowego należy zawsze pamiętać o jego ostatecznym przeznaczeniu oraz o użytkowniku końcowym.

W modelu warstwowym architektury komputerowej system operacyjny pełni szczególną rolę, zarządzając granicą między aplikacjami a warstwą fizyczną. W celu zapewnienia poprawnego i przewidywalnego działania systemu wprowadza poziomy abstrakcji, które są niezbędne z kilku istotnych powodów:

1. **Konieczność zapewnienia bezpieczeństwa** - Współczesne systemy operacyjne charakteryzują się ogromną złożonością, a ich kod źródłowy sięga kilkudziesięciu milionów linii [1]. Izolacja poszczególnych poziomów wprowadza wyraźną hierarchię uprawnień. Dzięki temu błąd powstały w kodzie aplikacyjnym pozostaje odizolowany i nie wpływa bezpośrednio na stabilność pozostałych funkcjonalności systemowych ani na pracę samego sprzętu.
2. **Zapewnienie przenoszalności oprogramowania** - Zastosowanie warstw pośrednich sprawia, że oprogramowanie nie jest bezpośrednio uzależnione od konkretnych urządzeń fizycznych. Pozwala to na uruchamianie tych samych programów na różnorodnych platformach sprzętowych, umożliwiając przy tym niezależny rozwój warstwy aplikacyjnej i sprzętowej.
3. **Umożliwienie korzystania z narzędzi wysokiego poziomu** - Ukrycie technicznych szczegółów pracy ze sterownikami lub rejestrami pozwala na tworzenie kodu w językach wysokiego poziomu. Programista może skupić się na logice i strukturze zgodnej ze sposobem myślenia człowieka, zamiast na bezpośredniej, niskopoziomowej obsłudze układów elektronicznych.

2.2.2. Podział systemów operacyjnych ze względu na strukturę systemu

Systemy operacyjne można klasyfikować według wielu kryteriów, z których jednym z najczęściej spotykanym jest podział, ze względu na ich docelowe przeznaczenie. Przy takim podziale możemy wyróżnić systemy operacyjne generalnego przeznaczenia, systemy operacyjne rozproszone, systemy operacyjne komputerów nmainframe, systemy operacyjne serwerów oraz wiele innych [1].

Jednakże, w kontekście analizy wydajności kluczowy jest podział ze względu na strukturę systemu, która określa architekturę jego jądra. Na tym etapie jądro systemu możemy zdefiniować jedyny program, który działa w pamięci komputera nieprzerwanie przez cały czas od jego uruchomienia, pełniąc funkcję pomostu komunikacyjnego między uruchomionymi aplikacjami, a fizycznym sprzętem [3].

Konstrukcja jądra decyduje o sposobie komunikacji między usługami systemowymi, a warstwą aplikacyjną, co bezpośrednio wpływa na narzuty czasowe oraz efektywność optymalizacji kodu programów. Z uwagi na to, że przedmiotem badań w niniejszej pracy jest system Linux, analiza skupi się głównie na architekturze monolitycznej. Dla celów porównawczych zostanie ona zestawiona z inną szeroko spotykaną architekturą w systemach wbudowanych - strukturą mikrojądra.

W **architekturze monolitycznej** (ang. *monolithic kernel*) wszystkie główne zadania systemu operacyjnego są połączone w jeden rozbudowany program komputerowy [1]. Zarządzanie pamięcią, programami oraz wszystkimi sterownikami urządzeń odbywa się we wspólnej przestrzeni. Takie podejście sprawia, że jądro zawiera w sobie ogromną liczbę elementów, co mocno

komplikuje proces jego tworzenia oraz późniejszego szukania błędów. Największą zaletą architektury monolitycznej jest jednak wysoka prędkość przetwarzania informacji – z uwagi na to, że jądro jest w całości kompilowane do jednego programu, poszczególne funkcjonalności mogą z siebie nawzajem korzystać, co znacząco przyspiesza działanie systemu. Przy architekturze monolitycznej aplikacje użytkownika są uruchamiane jako osobne procesy w chronionej pamięci. Dzięki temu jądro nie jest narażone na błędy w programach użytkownika, a różne aplikacje nie mogą nawzajem bezpośrednio mieć dostępu zasobów im nie przydzielonych. Główną wadą systemów monolitycznych pozostaje jednak słaba odporność na awarie samego jądra; jeśli błąd pojawi się w jego obrębie niego, na przykład w jednym ze sterowników, może dojść do awarii całego systemu.

Architektura mikrojądra (ang. *microkernel*) opiera się na zasadzie minimalizacji kodu wykonywanego z najwyższymi uprawnieniami procesora [3]. Kompletny system operacyjny składa się w tym przypadku z uproszczonego jądra oraz zestawu współpracujących ze sobą procesów systemowych. Samo mikrojądro realizuje jedynie fundamentalne zadania, takie jak zarządzanie wątkami oraz obsługa komunikacji międzyprocesowej [1]. Wszystkie pozostałe usługi, wliczając w to sterowniki urządzeń i systemy plików, są odseparowane od jądra i funkcjonują jako niezależne programy w przestrzeni użytkownika.

Główną zaletą takiego podejścia jest wysoka stabilność środowiska – awaria oprogramowania lub sterownika nie narusza integralności samego jądra. System potrafi zrestartować uszkodzony proces bez zaburzania pracy całego systemu. Wadą tego rozwiązania jest jednak niższa wydajność, wynikająca z narzutu czasowego na ciągłą konieczność komunikacji międzyprocesowej oraz częste przełączanie kontekstu procesora.

2.2.3. Korzyści i niedogodności płynące z stosowania systemów operacyjnych

2.3. Studium przypadku systemu Linux

2.3.1. Architektura systemu i jego jądro

2.3.2. Zarządzanie pamięcią

Mechanizm stronicowania pamięci

Wykorzystanie pamięci podręcznej procesora

2.3.3. Procesy i wątki w systemie Linux

2.3.4. Szeregowani zadań i determinizm czasowy w systemie Linux

2.3.5. Wykorzystanie systemu Linux w systemach wbudowanych

3. WYBRANY ALGORYTM PRZETWARZANIA DANYCH I TECHNIKI OPTYMALIZACJI KODU

- 3.1. *Teoretyczne podstawy analizy algorytmów***
- 3.2. *Opis i analiza Szybkiej Transformaty Fouriera***
- 3.3. *Warianty algorytmu Szybkiej Transformaty Fouriera***
- 3.4. *Przetworzenie kodu programu komputerowego na kod wykonywalny***
- 3.5. *Opis kompilacji programów w języku C***
- 3.6. *Poziomy i flagi optymalizacyjne kompilatora GCC***
- 3.7. *Zaawansowane techniki optymalizacji kodu***

4. TECHNIKI OPTYMALIZACJI PROGRAMÓW KOMPUTEROWYCH

4.1. Opis procesu kompilacji programu komputerowego

4.2. Programowe techniki optymalizacji programów komputerowych

4.3. Przegląd najpopularniejszych kompilatorów języka C

4.4. Poziomy optymalizacji dostępne w kompilatorach Clang i Gcc

4.5. Weryfikacja poprawności obliczeń z biblioteka zewnętrzna

5. PROJEKT SYSTEMU TESTOWEGO I METODYKA PRZEPROWADZONYCH BADAŃ

5.1. Założenia projektowe i architektura aplikacji

5.2. Opis platformy sprzętowej

5.3. Użyte narzędzia programistyczne

5.4. Metodyka przeprowadzonych pomiarów

5.5. Prezentacja przygotowywanych wariantów programu

5.6. Implementacja wersji wielowątkowej, opis biblioteki pthread

5.7. Weryfikacja poprawności obliczeń z biblioteka zewnętrzna

6. ANALIZA WYNIKÓW PRZEPROWADZONYCH BADAN

6.1. *Analiza wpływu flag kompilatora na kod sekwencyjny*

6.2. *Analiza wpływu optymalizacji ręcznej programu sekwencyjnego*

6.3. *Analiza skalowalności rozwiązania wielowątkowego*

6.4. *Zestawienie zbiorcze i dyskusja wyników*

7. PODSUMOWANIE I WNIOSKI

7.1. Wnioski koncowe

7.2. kierunek dalszych badan

SPIS RYSUNKÓW

2.1	Zestawienie przebiegów funkcji przydatności odpowiedzi $y(t)$ w dziedzinie czasu [1].	10
2.2	Warstwowy model systemu komputerowego	12

SPIS TABEL

BIBLIOGRAFIA

- [1] Andrew S. Tanenbaum and Herbert Bos. *Systemy operacyjne*. Helion, Gliwice, 2024. Przekład: Radosław Meryk.
- [2] K. C Wang. *Embedded and Real-Time Operating Systems*. Springer Nature, Cham, 1 edition, 2017.
- [3] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. J. Wiley & Sons, Hoboken, N.J., 8th ed. edition, 2008.